

Cybersecurity projects to rebuild

Homelab, security hardening and custom proxy

A shareable introduction to the original project ideas and the separate classroom builds. Read this first, then use the detailed handbook and repository to try an exercise yourself.

PRIVATE SERVICES

HARDENING

PRIVACY ROUTING

01 | A private, self-hosted homelab

Linux, Docker and NetBird

Original work: a small Debian host running self-hosted services, reached through a self-hosted NetBird network. **Classroom build:** a separate lab host, application networks and private administrator access. Learn what is listening, which sources are allowed, where data persists and what privileges Docker gives. Start with handbook chapters 01-03. [P01]

Caddy provides the private HTTPS endpoint. The default lab publishes only loopback HTTPS, reached through an authenticated SSH forward. Learners verify the lab CA, correct hostname and an actual denied connection; they do not make sensitive services public just to finish the exercise.

Passwords, files and photos

Vaultwarden: create a disposable vault, enrol the first user deliberately, close registration, keep server administration disabled, and practise two-factor authentication and recovery. Never import real passwords into a class lab.

Nextcloud: deploy separate application, PostgreSQL, Redis and scheduled-job components. Test ordinary-user permissions, a quota, correct reverse-proxy handling and file persistence. **Immich:** use matching official release files for an independent photo library and verify the media-plus-database recovery set. These reproduce the original project ideas without sharing its accounts, files or photos. Chapters 04-06. [P01]

The security outcome

The aim is not a screenshot of several green containers. A useful result proves that the right person can reach the right service, the wrong source or account cannot obtain private data, and the service's state survives the intended restart/recovery scenario. Record observed results, not copied expectations.

02 | Hardening and verification

Private administration and least privilege

The original work reported NetBird-only SSH, closed Vaultwarden registration, a disabled server-admin panel, two-factor setup and separation of selected public apps from private management. The pack turns those goals into checks without claiming a new audit of the original servers. [P01]

Chapter 08 includes a key-based SSH workflow with lockout precautions, effective-configuration checks, a narrow NetBird-only SSH guard and container port/privilege review. The exact snippets and guards are **new classroom examples**, not copied historical settings.

Monitoring, logs and DNS

Chapter 07 uses Uptime Kuma for a real check and a controlled failure/recovery. It explains private log review, the Docker-socket risk around monitoring tools, and a one-client AdGuard DNS-policy exercise. A monitoring success is not a security certification, and DNS filtering is not a malware verdict.

Chapter 10 separates public application ingress from private administration. It explains Cloudflare Tunnel versus Access, API-client compatibility, direct origin exposure and explicit allow/deny testing. The private core is the baseline; public-ingress work is a separately verified extension.

Recovery as a security control

Chapter 11 provides an encrypted core-stack backup and an isolated restore drill. The student checks logins, permissions and restored file contents. A second folder on the source disk, a quota or a synchronised client does not prove recovery after losing that disk. Independent Immich, NetBird and proxy state require their own recovery plans; the core script does not cover them.

03 | The custom privacy proxy

Separate private routing from internet egress

Original work: an Xray/REALITY proxy with Mullvad internet egress and private NetBird routing, using separate management/proxy VPS roles. **Classroom build:** fresh client/server profiles, a selective WireGuard path, a limited private-peer rule and explicit interruption tests. Chapter 09. [P01]

```

Test application -> local SOCKS -> Xray/REALITY proxy
                        |           |
                    private peer  Mullvad egress
                    via NetBird   via WireGuard

```

The repository supplies Xray client/server JSON templates, a selective WireGuard profile, a narrow egress guard, an unprivileged service unit and an owned TLS-target configuration. Every identity/key field is generated or filled locally; no usable personal proxy profile is included.

What classmates verify

Prove a public request takes the intended provider path. Prove only the permitted private test peer and port are reachable through the proxy. Interrupt WireGuard, the test peer and the local client separately; record what fails, what still works and what recovers. Never disconnect the only management path.

This baseline uses explicit TCP application proxying. It is not a tested system-wide TUN deployment, DNS/IPv6 leak audit or universal kill switch. Windows/Android full-device routing is an advanced extension requiring its own bootstrap, DNS, UDP, IPv6 and reconnect tests. The original working client configuration is not replaced by these examples.

The method teaches authorised network engineering and privacy boundaries, not promises of anonymity or evasion of the network owner's rules.

04 | Build, test and share

Choose a starting point

Learning route	Build first	Evidence
Private services	Chapters 01-03, then one app	Verified HTTPS, login and denied access
Security hardening	Chapters 07-08 and 10-11	Effective settings, failure tests and recovery
Custom proxy	Chapter 02, then 09	Correct egress, narrow private route and interruption results

A fresh local VM is enough to start the core. The self-hosted NetBird controller and advanced proxy need additional resources/accounts and may incur costs. Do not use a real password vault, personal photo library or college production network as the exercise target.

From the repository root, run the included example checks:

```
python3 -m unittest discover -s tests -v
python3 scripts/check_proxy.py labs/proxy/server.json.example
```

The revised helper suite passed **28 tests** during preparation. Those are local configuration/helper tests, not successful deployments. The full record is in `VALIDATION.md`.

Important validation boundary

The authoring checks do not establish a working Xray/REALITY handshake, WireGuard egress, nftables enforcement, remote-client setup, application login or successful backup restoration. Those require owned lab infrastructure and are marked as student verification tasks. Each selected Xray core must also pass its own configuration parser before startup.

Public learning material, private infrastructure

Production domains, server addresses, account handles, credentials, proxy subscription links, private keys, original screenshots and media are excluded. Use chapter 12 to publish a fresh repository without importing private Git history. No remote repository or live server was changed for this edition.

Follow course attribution/assistance rules. New teaching material is MIT-licensed; upstream applications retain their licenses. Technical sources and the meaning of reported original work are listed in `SOURCES.md` and chapter 00.